

Aula 02 - Protocolo HTTP e Mensagens de Retorno

Descrição: Exploração prática da estrutura de requisições e respostas HTTP (headers, body JSON, status codes) utilizando o navegador para inspecionar o tráfego de mensagens.

Introdução

Hoje vamos aprender de forma detalhada, simples e muito didática como funciona o **Protocolo HTTP** e a estrutura das mensagens que trafegam pela internet.

Antes de começarmos, é importante saber que, para entender o HTTP na prática, não podemos usar simplesmente arquivos locais salvos no computador (com o protocolo `file://`), pois o HTTP e o HTTPS são protocolos específicos para comunicação em rede.

Vamos desvendar esse assunto dividindo-o em três grandes partes: a **Requisição** (a pergunta do cliente), a **Resposta** (o retorno do servidor) e a **Inspeção de Tráfego** (como enxergar isso na tela).

Guia Completo: Protocolo HTTP e Mensagens de Retorno

Componente	Função Principal	Principais Elementos
Requisição (Request)	Mensagem enviada pelo cliente solicitando uma ação ou recurso.	Método HTTP, URL, Headers de pedido e Corpo (Body).
Resposta (Response)	Retorno do servidor com o resultado do processamento.	Linha de Estado (Status Code), Response Headers e Corpo (Body).
Inspeção de Tráfego	Análise em tempo real do fluxo de dados via ferramentas de desenvolvimento.	DevTools (Aba Network/Rede), Fiddler ou Proxies.

- **Regra Importante de Execução:** Para testar e inspecionar o protocolo HTTP/HTTPS na prática, é obrigatório utilizar um ambiente de rede (local ou remoto). Arquivos abertos localmente via protocolo `file://` não executam a arquitetura de comunicação do HTTP.

Módulo 1: Fundamentos e Arquitetura do Protocolo HTTP

Conceito e Camada de Aplicação

- O **HTTP (Hypertext Transfer Protocol)** opera na camada de aplicação da pilha de protocolos e atua como a espinha dorsal para a troca de dados na web.
- Define as regras formais e padronizadas para navegadores e aplicações solicitarem documentos e enviarem dados de volta ao servidor.
- Abstrai a complexidade das camadas de rede inferiores ao seguir especificações semânticas e sintáticas bem definidas, como a **RFC 9110**.

Modelo Cliente-Servidor e Abstração do Navegador

- **Fluxo Estritamente Assíncrono/Iniciado pelo Cliente:** Toda comunicação começa no cliente (navegador

ou app), que envia uma **Requisição** quando precisa de um recurso ou ação. O servidor processa o pedido e devolve uma **Resposta**.

- **Gerenciamento Automatizado:** APIs do navegador (como XMLHttpRequest ou a API Fetch) gerenciam os detalhes operacionais de baixo nível do protocolo em segundo plano — manipulando automaticamente *cookies*, redirecionamentos, *proxies* e políticas de *cache*.

Importância da Interoperabilidade Universal

- O HTTP provê um padrão universal que garante que sistemas heterogêneos (escritos em linguagens diferentes ou rodando em sistemas operacionais distintos) conversem sem falhas de interpretação.
 - Suporta a troca fluida de múltiplos formatos de mídia e dados: páginas HTML, arquivos CSS, scripts JS, imagens, além de dados estruturados como XML e JSON em chamadas Ajax.
-

Módulo 2: Anatomia da Requisição HTTP (Request)

Uma requisição HTTP enviada pelo cliente é composta estruturalmente por 4 partes essenciais: Método/Verbo, URL, Cabeçalhos de Pedido (*Request Headers*) e Corpo (*Request Body*).

1. Métodos / Verbos HTTP

Os verbos definem o tipo de ação solicitada ao servidor. Embora existam múltiplos métodos — como PUT, DELETE, HEAD e OPTIONS —, os dois mais utilizados no início do desenvolvimento são:

- **GET (Obter):**
 - **Finalidade:** Buscar ou ler dados do servidor sem gerar efeitos colaterais.
 - **Parâmetros:** São anexados diretamente na URL após o caractere ? (ex.: ?find=pizza&zipcode=02134).
 - **Característica:** Respostas a requisições GET podem ser salvas em *cache*. Navegadores legados (como o Internet Explorer) limitavam o tamanho total da URL a 2.048 caracteres.
- **POST (Enviar):**
 - **Finalidade:** Enviar dados estruturados para processamento ou gravação (ex.: formulários ou cadastros no banco de dados).
 - **Parâmetros:** São transportados ocultos dentro do corpo (*body*) da mensagem, permitindo volumes massivos de dados.

2. URL (Uniform Resource Locator)

Funciona como o endereço exato que identifica o recurso no servidor. Pode ser resolvida de forma relativa ao documento base e determina a rota de acesso.

3. Cabeçalhos de Requisição (Request Headers)

Pares de chave-valor que enviam metadados sobre o contexto da requisição e negociação de conteúdo:

- **Content-Type:** Informa ao servidor o formato do corpo enviado (ex.: text/plain, application/json ou application/x-www-form-urlencoded).
- **Accept-Encoding:** Informa quais algoritmos de compressão o navegador suporta (ex.: gzip, que pode reduzir o tamanho dos arquivos em até 70% na transferência).
- **User-Agent:** Identifica o navegador, versão e sistema operacional do cliente.
- **Referer:** Registra a URL da página anterior que continha o link que gerou a requisição.

- **Cabeçalhos Automáticos:** O navegador preenche de forma automática e segura campos críticos como Content-Length, Date e Cookie.

4. Corpo da Requisição (Request Body)

- **Em requisições GET:** O corpo viaja vazio (ou nulo).
- **Em requisições POST:** Contém os dados da requisição. Atualmente, o padrão mais adotado para APIs é o **JSON** (*JavaScript Object Notation*), que organiza as informações em pares chave/valor:

```
{  
  "find": "pizza",  
  "zipcode": 02134  
}
```

Módulo 3: Anatomia da Resposta HTTP (Response)

A resposta enviada pelo servidor é dividida em 3 partes: Linha de Estado, Cabeçalhos de Resposta (*Response Headers*) e Corpo da Resposta (*Response Body*).

1. Códigos de Status (Status Code)

Códigos numéricos de três dígitos acompanhados de uma mensagem textual explicativa:

CLASSES DE CÓDIGOS HTTP		
1xx	Informativas	Requisição recebida e em processamento.
2xx	Sucesso	Ação recebida, entendida e aceita.
3xx	Redirecionamento	Ações adicionais necessárias (ou uso Cache).
4xx	Erro do Cliente	Requisição sintaticamente incorreta ou inválida
5xx	Erro do Servidor	O servidor falhou ao processar pedido válido

- **Família 1xx (Informativas):** 100 Continue, 101 Switching Protocols.
- **Família 2xx (Sucesso):**
 - 200 OK: Requisição bem-sucedida; o corpo contém o resultado.
 - 204 No Content: Processado com sucesso, mas a resposta intencionalmente não possui corpo.
- **Família 3xx (Redirecionamento / Cache):**
 - 302 Found: O recurso solicitado está temporariamente em outra URL.
 - 304 Not Modified: Indica que o recurso não mudou desde o último acesso. O navegador reaproveita a cópia guardada em *cache* local, economizando largura de banda.
- **Família 4xx (Erro do Cliente):**
 - 404 Not Found: A URL solicitada não corresponde a nenhum recurso no servidor.
- **Família 5xx (Erro do Servidor):**
 - 500 Internal Server Error: Falha genérica no código do servidor.

- Outros comuns: 502 Bad Gateway, 503 Service Unavailable, 504 Gateway Timeout.

2. Cabeçalhos de Resposta (Response Headers) em Detalhe

Metadados essenciais que instruem o cliente sobre como interpretar e armazenar o conteúdo:

- **Tipos MIME (Content-Type):**

- Orienta o parseamento do corpo. Se omitido ou configurado incorretamente pelo servidor, a análise pode falhar no cliente.
- Bibliotecas como o **jQuery** inspecionam essa string para decidir como processar o retorno (convertendo automaticamente para XML se tiver "xml", para JSON se tiver "json" ou executando o script se contiver "javascript").
- Métodos como `overrideMimeType()` permitem que o script do cliente sobrescreva a interpretação do tipo MIME caso o servidor envie o cabeçalho errado.
- Esquemas como `blob://` emitem automaticamente um Content-Type baseado no tipo do objeto na memória.

- **Data de Modificação (Last-Modified):**

- Especifica o momento da última alteração do arquivo. Em documentos HTML, pode ser lido via `document.lastModified`.
- Permite validação condicional: enviando a opção `ifModified`, o cliente armazena o valor e realiza a próxima requisição com o cabeçalho `If-Modified-Since`, recebendo um 304 Not Modified caso o arquivo continue idêntico.

- **Diretivas de Cache (Expires):**

- Define uma data/hora absoluta (no formato GMT / RFC 1123) limite para a validade do arquivo no *cache*.
- Para arquivos estáticos que nunca mudam, é padrão configurar um Expires de futuro distante (limitado a no máximo 1 ano pelas normas HTTP).

3. Corpo da Resposta (Response Body) em Detalhe

É o *payload* real retornado após os cabeçalhos:

- **Documentos HTML:** Traz a estrutura visual renderizada pelo navegador para formar a árvore DOM.
- **Objetos JSON:** Usados em APIs para envio de dados estruturados.
- **Erros Padronizados (RFC 9457):** Retornos de erro estruturados sob a especificação `application/problem+json`, contendo os campos `type`, `status`, `title`, `detail` e `instance`.
- **Arquivos Binários e Blobs:** Retorna dados de imagens, arquivos compactados ou áudio/vídeo, que podem ser manipulados como `Blob` ou `ArrayBuffer`.
- **Leitura Assíncrona e Stream:** Em chamadas assíncronas (XHR/Fetch), o corpo pode ser lido via propriedade `responseText` ou consumido em múltiplos pedaços (*chunks*) via *streaming* à medida que chega.

Módulo 4: Prática de Inspeção e Exercícios de Fixação

Exercício Guiado: Inspeção Prática de Tráfego no Navegador (Passo a Passo)

Siga as etapas abaixo para inspecionar requisições e respostas HTTP em tempo real utilizando o navegador:

1. **Abertura do Painel de Desenvolvedor:** Abra o navegador Web, acesse um site qualquer (lembrando de

utilizar requisições HTTP/HTTPS em vez do protocolo `file://`) e pressione **F12** para abrir o DevTools.

2. **Navegação até a Aba de Rede:** Clique na aba **Rede (Network)** e certifique-se de que a gravação de tráfego está ativa.
3. **Seleção da Requisição:** Recarregue a página (**F5**). Selecione a primeira requisição da lista na cascata para abrir o painel de detalhes.
4. **Verificação de Cabeçalhos (Headers):**
 - o Em **General**, identifique o **Método HTTP** (ex.: `GET`) e o **Código de Status** (ex.: `200 OK`).
 - o Em **Response Headers**, localize o `Content-Type` (ex.: `text/html`).
 - o Em **Request Headers**, localize o `User-Agent`.
5. **Inspeção do Corpo e Desempenho:** Clique na aba **Response** (ou **Preview**) para examinar o corpo (*body*) retornado e compare o tamanho do download com o tempo gasto na transferência.

Atividade Individual: Inspeção Prática e Mapeamento Semântico

Atenção: Cada estudante deve realizar a sua própria análise e guardar os resultados para a montagem do relatório final.

• Parte A — Inspeção Individual de Rede:

1. Escolha um site de sua preferência e inspecione a requisição principal via DevTools.
2. Registre: a URL acessada, o **Método HTTP**, o **Código de Status**, o cabeçalho `Content-Type` e o `User-Agent`.
3. Compare e anote a relação entre o tamanho total da resposta e o tempo gasto no download.

- **Parte B — Mapeamento de Cenários:** Associe as situações da **Coluna A** ao código de estado correspondente na **Coluna B**:

Coluna A (Cenários Reais)
1. O usuário tentou acessar uma página cuja URL foi digitada incorretamente e não existe.
2. O cadastro de um novo produto foi efetuado com sucesso no banco de dados.
3. Um cliente tentou enviar um arquivo de vídeo de 5GB, excedendo o limite permitido pelo servidor.
4. O servidor de banco de dados caiu inesperadamente enquanto processava a requisição.
5. Uma requisição condicional identificou que o recurso não foi alterado desde o último acesso e pode ser lido do cache.
Coluna B (Códigos de Status)
() <code>304 Not Modified</code>
() <code>201 Created</code>
() <code>404 Not Found</code>
() <code>413 Content Too Large</code>
() <code>500 Internal Server Error</code>

Atividade em Dupla: Diagnóstico de Erros via JSON (RFC 9457)

Em duplas, analisem a estrutura da mensagem de erro padronizada (`application/problem+json`) enviada por uma API e respondam às questões de diagnóstico:

```
{  
  "type": "https://api.loja.com/erros/saldo-insuficiente",  
  "status": 400,  
  "title": "Falha no Pagamento",  
  "detail": "Sua conta não possui saldo suficiente para concluir o pedido de R$ 150,00.",  
  "instance": "/transacoes/tx-987654"  
}
```

1. **Análise do Status:** Qual é a classe e o significado técnico do código de status informado no campo `status`?
2. **Diferenciação Semântica:** Qual é a diferença prática entre as informações fornecidas no campo `title` e no campo `detail`?
3. **Rastreabilidade:** Para que serve o campo `instance` dentro dessa estrutura de erro?

Instruções para Entrega

A entrega das atividades deve ser feita em **arquivo único individual**, contendo os seguintes componentes compilados:

1. **Relatório da Atividade Individual:** A ficha de inspeção de rede (Parte A) e o gabarito do mapeamento semântico (Parte B).
2. **Resolução da Atividade em Dupla:** As respostas discursivas elaboradas em conjunto para o diagnóstico do JSON de erro (RFC 9457).